

# Deploying OneAgent

---

**Series:** ONBRD | **Notebook:** 5 of 10 | **Created:** December 2025 | **Last Updated:** 2026-01-20

## Getting Data Into Dynatrace

---

OneAgent is the foundation of Dynatrace monitoring. This notebook covers deployment strategies, installation methods, and verification steps to ensure your infrastructure is reporting data.

---

## Table of Contents

---

1. What is OneAgent?
  2. Deployment Strategy
  3. Generating a Deployment Token
  4. Installation Methods
  5. Kubernetes Deployment
  6. Verifying Deployment
  7. Troubleshooting
  8. Next Steps
- 

## Prerequisites

---

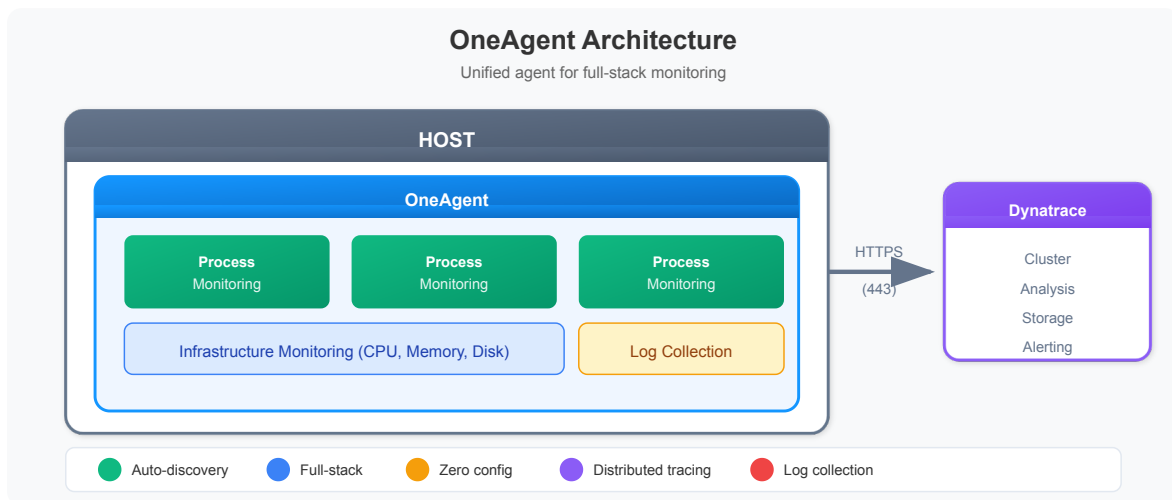
- Admin access to Dynatrace environment
- API token with `InstallerDownload` scope (created in ONBRD-02)
- Root/admin access to target hosts

- Network access from hosts to Dynatrace (port 443) or ActiveGate (ONBRD-03)
- Cloud integrations configured (ONBRD-04) for full context

## 1. What is OneAgent?

OneAgent is Dynatrace's unified monitoring agent that:

| Capability                 | Description                                      |
|----------------------------|--------------------------------------------------|
| <b>Auto-discovery</b>      | Automatically detects hosts, processes, services |
| <b>Full-stack</b>          | Infrastructure, application, and user experience |
| <b>Zero configuration</b>  | Works immediately after installation             |
| <b>Distributed tracing</b> | End-to-end transaction visibility                |
| <b>Log collection</b>      | Collects and forwards log data                   |



## 2. Deployment Strategy

### Phased Rollout (Recommended)

Don't deploy everywhere at once. Use a phased approach:

| Phase             | Scope                  | Purpose                                   |
|-------------------|------------------------|-------------------------------------------|
| <b>Pilot</b>      | 2-5 non-critical hosts | Validate installation, network, discovery |
| <b>Expand</b>     | One application tier   | Test service detection, tracing           |
| <b>Production</b> | Full environment       | Complete coverage                         |

## Deployment Methods by Environment

| Environment           | Recommended Method                |
|-----------------------|-----------------------------------|
| <b>Bare metal/VMs</b> | Direct install or package manager |
| <b>Kubernetes</b>     | Dynatrace Operator                |
| <b>OpenShift</b>      | Dynatrace Operator                |
| <b>AWS ECS</b>        | Task definition sidecar           |
| <b>Azure</b>          | VM Extension or AKS Operator      |
| <b>GCP</b>            | Direct install or GKE Operator    |

## Network Requirements

OneAgent needs outbound HTTPS (443) to:

- \*.apps.dynatrace.com (SaaS)

If direct access isn't possible, OneAgents connect through **ActiveGate** (see ONBRD-03).

## 3. Generating a Deployment Token

You need an API token with installer download permissions.

**Location:** Account Management → Access tokens → Generate new token

## Required Scope

| Scope                                 | API Name          | Purpose                     |
|---------------------------------------|-------------------|-----------------------------|
| PaaS integration - Installer download | InstallerDownload | Download OneAgent installer |

## Token Naming Convention

Use descriptive names:

- prod-oneagent-deployment
- dev-installer-token
- k8s-operator-token

**Important:** Copy the token immediately after creation. It won't be shown again.

## 4. Installation Methods

### Linux (Direct Download)

```
# Download the installer
wget -O Dynatrace-OneAgent.sh \
  "https://{tenant-id}.apps.dynatrace.com/api/v1/deployment/installer/agent/unix/default/latest?
  Api-Token={token}&arch=x86&flavor=default"

# Run the installer
sudo /bin/sh Dynatrace-OneAgent.sh
```

### Linux (One-liner)

```
wget -O Dynatrace-OneAgent.sh "https://{tenant-id}.apps.dynatrace.com/api/v1/deployment/installer/agent/unix/default/latest?
  Api-Token={token}" && sudo /bin/sh Dynatrace-OneAgent.sh
```

## Windows (PowerShell)

```
# Download the installer
Invoke-WebRequest -Uri "https://{tenant-id}.apps.dynatrace.com/api/v1/deployment/installer/agent/windows/default/latest?Api-Token={token}" -OutFile Dynatrace-OneAgent.exe

# Run the installer
.\Dynatrace-OneAgent.exe
```

## Configuration Management

For Ansible, Puppet, Chef, or other tools, see:

- [Ansible Collection](#)
- [Puppet Module](#)
- [Chef Cookbook](#)

## 5. Kubernetes Deployment

---

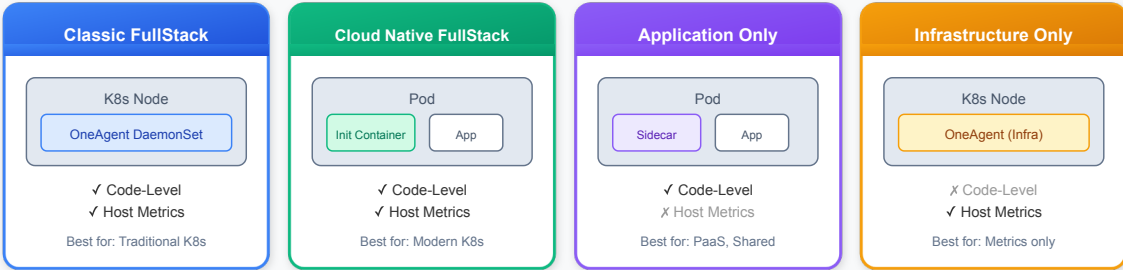
For Kubernetes environments, use the Dynatrace Operator—the recommended approach for deploying and managing OneAgent in containerized environments.

### OneAgent Deployment Modes

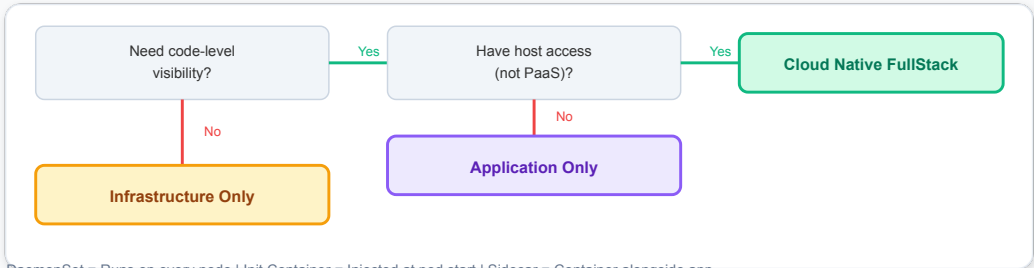
The Dynatrace Operator supports multiple deployment modes. Choose based on your requirements:

# OneAgent Kubernetes Deployment Modes

Choose the right mode for your Kubernetes environment



## Decision Guide



DaemonSet = Runs on every node | Init Container = Injected at pod start | Sidecar = Container alongside app

| Mode                   | Deployment           | Code-Level | Host Metrics | Best For                              |
|------------------------|----------------------|------------|--------------|---------------------------------------|
| Cloud Native FullStack | Init container + CSI | Yes        | Yes          | Modern K8s (recommended)              |
| Application Only       | Sidecar              | Yes        | No           | PaaS, shared nodes, OpenShift         |
| Host Monitoring        | DaemonSet            | No         | Yes          | When code-level monitoring not needed |

**Note:** Classic FullStack mode is deprecated. Dynatrace recommends migrating to Cloud Native FullStack for all Kubernetes environments.

# Deployment Method: Helm vs Manifests

| Method         | Pros                                               | Cons                    | Best For                             |
|----------------|----------------------------------------------------|-------------------------|--------------------------------------|
| Helm Charts    | Version management, easy upgrades, values override | Requires Helm           | Production, GitOps                   |
| Manifest Files | Simple, no dependencies                            | Manual version tracking | Air-gapped, constrained environments |

## Prerequisites

- `kubectl` access to cluster
- API token with required scopes
- Cluster admin permissions

## Required Token Scopes for Kubernetes

| Scope                          | Purpose                |
|--------------------------------|------------------------|
| <code>InstallerDownload</code> | Download OneAgent      |
| <code>entities.read</code>     | Read topology          |
| <code>settings.read</code>     | Read configuration     |
| <code>settings.write</code>    | Write configuration    |
| <code>DataExport</code>        | Export data (optional) |

## Installation Option 1: Helm (Recommended)

```
# Add Dynatrace Helm repository
helm repo add dynatrace
https://raw.githubusercontent.com/Dynatrace/dynatrace-
operator/main/config/helm/repos/stable
helm repo update

# Create namespace and secret
kubectl create namespace dynatrace
kubectl -n dynatrace create secret generic dynakube --from-
literal="apiToken=<API_TOKEN>" --from-
literal="dataIngestToken=<DATA_INGEST_TOKEN>"

# Install with Helm
helm install dynatrace-operator dynatrace/dynatrace-operator \
  --namespace dynatrace \
  --set installCRD=true
```

## Installation Option 2: Manifest Files

### 1. Navigate to Kubernetes monitoring

- Infrastructure → Kubernetes → Add cluster

### 2. Follow the wizard

- Select your distribution
- Choose observability options
- Download dynakube.yaml

### 3. Apply to cluster

```
bash kubectl apply -f https://github.com/Dynatrace/dynatrace-
operator/releases/download/v1.3.2/kubernetes.yaml kubectl apply -f
dynakube.yaml
```

### 4. Verify deployment

```
bash kubectl get pods -n dynatrace
```

## DynaKube Custom Resource Configuration

The DynaKube CR is the primary configuration for the Dynatrace Operator. Here are examples for each deployment mode:



**Important:** Use `apiVersion: dynatrace.com/v1beta5` for Dynatrace Operator 1.7.0+. Earlier versions (v1beta1, v1beta2) are deprecated and no longer supported.

### Cloud Native FullStack (Recommended for most K8s):

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Cloud Native FullStack – automatic init container injection
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/master
          operator: Exists
      args:
        - --set-host-group=my-k8s-cluster

  # ActiveGate for Kubernetes API monitoring
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
      - dynatrace-api
```

### Application Only (for PaaS or shared nodes):

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Application Only – sidecar injection, no host monitoring
  oneAgent:
    applicationMonitoring:
      useCSIDriver: true
```

### Host Monitoring (host metrics only):

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Host monitoring only – no code-level visibility
  oneAgent:
    hostMonitoring: {}
```

## Air-Gapped / Private Registry Deployment

For environments without internet access, host images in a private registry:

```

apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Custom image locations for air-gapped environments
  oneAgent:
    cloudNativeFullStack:
      image: my-registry.example.com/dynatrace/oneagent:latest

  activeGate:
    capabilities:
      - kubernetes-monitoring
    image: my-registry.example.com/dynatrace/activegate:latest

```

### Image Mirroring Script:

```

# Mirror required images to private registry
IMAGES=(
  "docker.io/dynatrace/dynatrace-operator:v1.3.2"
  "docker.io/dynatrace/dynatrace-oneagent:latest"
  "docker.io/dynatrace/dynatrace-activegate:latest"
)

for img in "${IMAGES[@]"; do
  docker pull $img
  docker tag $img my-registry.example.com/${img#*/}
  docker push my-registry.example.com/${img#*/}
done

```

## Kubernetes Metadata and Dynatrace Tags

Dynatrace can leverage Kubernetes labels and annotations for observability:

| Feature                 | Behavior                                         | Limitations                             |
|-------------------------|--------------------------------------------------|-----------------------------------------|
| <b>Pod labels</b>       | Appear as [Kubernetes] context tags on processes | Requires OneAgent code module injection |
| <b>Annotations</b>      | Available as custom metadata at Process level    | Requires view RBAC permissions          |
| <b>Namespace labels</b> | Available via Primary Grail tags                 | Does not enrich K8s metrics or events   |

#### Requirements:

- Service accounts need `view` access via rolebinding/clusterrolebinding
- Labels are read via Kubernetes REST API at deployment time
- Not all entities receive automatic tags (namespaces, pods, workloads have limited support)

For more control, configure automatic tagging rules: **Settings → Automatically applied tags**

## Multi-Tenant / Namespace Isolation

For large clusters with multiple teams, use namespace selectors:

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: team-a-dynakube
  namespace: dynatrace
spec:
  apiUrl: https://team-a-tenant.live.dynatrace.com/api

  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dynatrace-tenant: team-a
```

This allows different teams/tenants to monitor different namespaces within the same cluster.

## 6. Verifying Deployment

---

After installation, verify OneAgent is reporting data.

```
// Check all hosts with OneAgent
fetch dt.entity.host
| fields entity.name, state, monitoringMode
| filter state == "RUNNING"
| sort entity.name
| limit 50
```

```
// Check hosts by monitoring state
fetch dt.entity.host
| summarize host_count = count(), by: {state, monitoringMode}
| sort host_count desc
```

```
// Check hosts by monitoring state - useful for verifying deployment
fetch dt.entity.host
| fields entity.name, state, monitoringMode
| summarize host_count = count(), by: {state}
| sort host_count desc
```

```
// Check discovered services
fetch dt.entity.service
| fields entity.name, serviceType
| summarize service_count = count(), by: {serviceType}
| sort service_count desc
```

```
// Check for processes discovered
fetch dt.entity.process_group
| fields entity.name
| sort entity.name
| limit 50
```

### Host Verification Commands

**Linux:**

```
# Check OneAgent status
sudo systemctl status oneagent

# Check connection to Dynatrace
sudo /opt/dynatrace/oneagent/agent/tools/oneagent-connection-check

# Check OneAgent logs
sudo tail -100 /var/log/dynatrace/oneagent/oneagent.log
```

#### Windows:

```
# Check service status
Get-Service -Name "Dynatrace OneAgent"

# Check connection
&& "C:\Program Files\dynatrace\oneagent\agent\tools\oneagent-connection-check.exe"
```

## 7. Troubleshooting

### Common Issues

| Issue                    | Cause                   | Solution                             |
|--------------------------|-------------------------|--------------------------------------|
| Host not appearing       | Network blocked         | Check firewall rules for 443         |
| Host showing but no data | Agent not running       | Restart OneAgent service             |
| Services not discovered  | Processes not restarted | Restart monitored applications       |
| Old OneAgent version     | Auto-update disabled    | Enable auto-update or manual upgrade |
| Connection errors        | Proxy required          | Configure ActiveGate or proxy        |

## Network Verification

```
# Test connectivity to Dynatrace
curl -v https://{tenant-id}.apps.dynatrace.com/api/v1/time

# Check DNS resolution
nslookup {tenant-id}.apps.dynatrace.com

# Test with OpenSSL
openssl s_client -connect {tenant-id}.apps.dynatrace.com:443
```

## Process Restart Requirement

OneAgent injects into running processes. After initial installation:

- **New processes** - Automatically monitored
- **Existing processes** - Restart required for full monitoring

For deep code-level visibility, restart:

- Application servers (Tomcat, JBoss, etc.)
- Web servers (Apache, Nginx, IIS)
- .NET/Java applications
- Node.js applications

## 8. Next Steps

---

With OneAgent deployed and verified:

1. **ONBRD-06: Organizing Your Environment** - Set up tags, segments, and naming conventions
2. Wait 15-30 minutes for full topology discovery
3. Check Smartscape for service dependencies
4. Plan broader rollout based on pilot results

## Deployment Checklist

- [ ] Pilot hosts deployed (2-5 hosts)
- [ ] Hosts appearing in Dynatrace
- [ ] OneAgent status "RUNNING" on all hosts

- [ ] Services being discovered
  - [ ] Connection check passing
  - [ ] Smartscape showing topology
- 

## Summary

---

In this notebook, you learned:

- What OneAgent does and how it works
  - Phased deployment strategy
  - How to generate deployment tokens
  - Installation methods for Linux, Windows, and Kubernetes
  - How to verify successful deployment
  - Common troubleshooting steps
- 

## References

---

- [OneAgent Overview](#)
  - [Linux Installation](#)
  - [Windows Installation](#)
  - [Kubernetes Operator](#)
  - [Deployment API](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*